Lab 2 – C Programming

IS1200/IS1500 staff

2026

Contents

1 Introduction **2**
- 1.1 Preparations . . . 2
- 1.2 Resources . . . 2
- 1.3 Reading Guidelines . . . 3

2 Examination **3**

3 Assignments **3**
- 3.1 Assignment 1: Basic Control-Flow . . . 3
- 3.2 Assignment 2: Functions and Side Effects . . . 4
- 3.3 Assignment 3: Arrays . . . 5
- 3.4 Assignment 4: Pointers . . . 6
- 3.5 Assignment 5: Memory Layout . . . 7
- 3.6 Assignment 6: Surprise Assignment . . . 8

4 Version History **9**

1 Introduction

Welcome to the second lab in the course. In this lab, you will learn the basics of the C programming language. More specifically, after finishing this lab, you will be able to

1. Construct short C programs that include for-loops, if-statements, arrays, and pointers.
2. Analyze C programs and understand how a compiled program is laid out in memory.

1.1 Preparations

You must do the lab work in advance, except for a few specific tasks. The teachers will examine your skills and knowledge at the lab session.

> Before your booked lab session, upload your solution files to the Canvas assignment **Lab 2 Files + Upload code** – the assignment page lists exactly which files. When you log in on the KTH lab computers at the session, your submitted code is downloaded automatically, and you explain it to the teachers there. If you modify your code during the session, you may re-upload it to Canvas; re-uploads end up in the "Files" section of your group page.

Booking opens one week before each lab session. We recommend that you book as soon as booking opens, and start preparing at least a week before the session.

Assignments 1 through 5 must be prepared completely before the start of your lab session.

Assignment 6 (the surprise assignment) will be performed during your lab session.

You can ask for help any time before the lab session. There are several ways to get help; see the course website for details and alternatives.

During the lab session, the teachers will perform examinations, but can also offer help. Make sure to state clearly that you want to ***ask questions***, rather than being examined.

Sometimes, our teachers may have to refer help-seekers to other sources of information, so that other students can be examined.

NOTE: In this exercise, you MUST explicitly declare in the code whether you or your lab partner wrote the code. That is, you should explicitly state who typed on the computer. It is NOT allowed to copy ANY code from anyone else or from the Internet.

Out of the four first assignments, each of you (lab partners) should author two assignments each. This means that it is not allowed for just one of you to sit and type the code for all the assignments. You should still sit together and discuss all the problems and the solutions and the lab partner who is not typing should still be able to analyze the program and explain all the details of the solution.

It is also allowed that both of you create your own solutions, for all exercises. This is actually the best alternative. It is OK if your solutions are similar, as long as you have written them individually.

1.2 Resources

For Assignments 1 to 4, you may use a C compiler of your choice. We recommend using compiler such as *gcc* from the command line interface. See the course website for more information about lab resources.

For assignment 5, you will use a DTEK-V development board, which includes a RISC-V/32 processor. To access the lab environment, either using your own computer or through KTH computers, please visit the "Software Resources" page on Canvas.

The files for assignments 1–4 are attached to the Canvas assignment **Lab 2 Files + Upload code** as `lab2-files.zip` (the same assignment also carries `riscv32tests.zip`, used in Assignment 5). Unpack it; the bundle contains four files: `prime.c` (Assignment 1), `print-primes.c`

(Assignment 2), and `pointers.c` together with `pointers.S` (Assignment 4). Assignment 3 starts from `print-primes.c` and asks you to create two new files (`sieves.c` and `sieves-heap.c`) yourself; they are not in the download.

1.3 Reading Guidelines

Review the following course material while you are preparing your solutions.

- Lectures 1 and 4.
- The Harris & Harris (2021) book, Appendix C on C Programming.
- The online C programming tutorial links are available on the course webpage under *Literature and resources*.

2 Examination

The examination takes place during your booked lab session. The teachers examine your solutions at the lab computers: you run your programs, explain your code, and answer questions about it (the "Questions for Assignment" lists throughout this document show the kind of questions to expect). During the session you also solve a surprise assignment *as a pair* (see Assignment 6) and demonstrate the result to the teacher. The questions, however, are answered *individually*: the teacher addresses each of you in turn, and each of you must be able to answer on your own.

Both lab partners must be able to explain all parts of the solution. All programs must compile without errors and warnings before the examination. See the course Canvas page for booking rules and deadlines.

This lab is examined in three parts. You can only be examined on one part at a time.

- Part 1 – Assignments 1 to 3.
- Part 2 – Assignment 4.
- Part 3 – Assignments 5 and 6.

When you pass a part, the teacher enters your grade in Canvas at the session. **Before you leave, immediately double-check under *Grades* that the teacher has registered your result** – if it is missing, tell the teacher right away.

> **Remember:** upload your solution files to the Canvas assignment **Lab 2 Files + Upload code** *before* your booked session (the assignment page lists exactly which files). When you log in on the KTH lab computers, your submitted code is downloaded automatically – that is the code you present. If you modify it during the session, you may re-upload it to Canvas.

3 Assignments

3.1 Assignment 1: Basic Control-Flow

The purpose of this assignment is to learn how to write a simple C program that includes basic control structures, including if-statements and for-loops.

Assuming that you are using gcc as your compiler, compile and run the file as follows:

```
gcc prime.c -o prime
./prime
```

The program prints the number 0 three times.

Task: Implement the body of the function with this exact signature (already declared for you in `prime.c`):

```
int is_prime(int n)
```

The return type `int` means the function returns an integer; the single parameter `int n` is the number to test. Return `1` if `n` is a prime number and `0` otherwise. The function must behave correctly for all positive integer values of `n`, and the implementation should use only simple loops and `if`-statements (no arrays, no recursion).

3.2 Assignment 2: Functions and Side Effects

The purpose of this assignment is to learn the concepts of functions, arguments, global variables, and side effects.

Open the file `print-primes.c`. The function `print_primes` defined there includes example code that shows how to print a list of prime numbers to the standard output.

A note on `printf` format specifiers. The example uses the format string `"%10d "`. The `%d` part says "print a signed integer in decimal"; the `10` between `%` and `d` is a *minimum field width*: if the integer takes fewer than 10 characters, `printf` pads it on the left with spaces, so successive calls line up neatly into columns. A trailing literal space after the `%10d` separates columns visually. The `COLUMNS` macro (set to 6 in the template) decides how many numbers fit on a row before a newline is printed.

Task: You should now do the following:

1. Create a helper function with this exact signature:

   ```
   void print_number(int n)
   ```

   The keyword `void` (before the function name) means the function returns nothing. The single parameter `int n` is the integer the caller wants printed.

   The body must do two things:

   (a) Print `n` with the format `"%10d "` (10-character-wide, right-aligned, followed by a space) so successive numbers line up into columns:

   ```
   printf("%10d ", n);
   ```

   (b) Wrap the table to a new row after every `COLUMNS` numbers. Notice that `print_number` receives only one input, the integer `n`, so it cannot decide from its arguments alone whether the current call is the one that should be followed by a newline. It would need to know "how many numbers have I printed so far on this row?", and that information is not in `n`. Something has to remember the position between calls. The simplest way is a *global variable*: declare an `int` at the top of `print-primes.c`, *outside any function*, right next to the `#define COLUMNS` line:

   ```
   int counter = 0;
   ```

   Because it is declared outside any function, every call to `print_number` sees the same `counter` and can update it. After each `printf`, increment `counter`; once it reaches `COLUMNS`, print a newline (`printf("\n")`) and reset `counter` to `0`.

   (Using a global variable like this is convenient but is generally considered poor practice; we will discuss why in the oral session.)

2. Insert your `is_prime` function from the previous exercise. Then replace the body of `print_primes` so that it prints every prime number from 2 up to `n` by calling `is_prime` to test each candidate and `print_number` to print the ones that are prime. The skeleton you need to fill in looks like this:

```
void print_primes(int n)
{
    // ... your code goes here ...
    // loop from 2 to n, and for each candidate i,
    // call is_prime(i); if it returns 1, call print_number(i).
}
```

If you execute the program as follows:

```
./print-primes 105
```

the output should be all prime numbers up to and including 105, with each number occupying 10 characters and 6 numbers per row:

```
         2         3         5         7        11        13
        17        19        23        29        31        37
        41        43        47        53        59        61
        67        71        73        79        83        89
        97       101       103
```

Questions for Assignment 2 Before the oral exam, you should prepare the answers to the following questions. You will need to be able to answer these questions to pass the assignment.

- What does it mean when a function does not return a value? How do you state that in a program? How can the function (or more precisely, the procedure) perform anything useful?
- How did you implement the side effect needed to make *print_number* behave correctly?

3.3 Assignment 3: Arrays

The purpose of this assignment is to learn how to use arrays in a program.

Task 1: Your task is now to create a new source file called `sieves.c`. Start by copying `main` and the `print_number` helper you wrote in Assignment 2 (the function that prints one integer in the `%10d` format and wraps to a new row after `COLUMNS` values) from your `print-primes.c` into this new file.

Replace the call to `print_primes` with a call to a new function with this exact signature:

```
void print_sieves(int n)
```

The single parameter `int n` is the upper bound: the function should print every prime number from 2 up to and including `n`. The output for a given input must match Assignment 2 exactly.

The algorithm you must follow. The implementation of `print_sieves` must use the *Sieve of Eratosthenes*, an algorithm dating back to ancient Greece. The Wikipedia page describes it well: https://en.wikipedia.org/wiki/Sieve_of_Eratosthenes. The version you must implement is the textbook one given there:

1. Create a list of consecutive integers from 2 through `n`: `(2, 3, 4, ..., n)`.
2. Initially, let `p` equal 2, the smallest prime number.

3. Enumerate the multiples of `p` by counting in increments of `p` from `2p` to `n`, and mark them in the list (these will be `2p, 3p, 4p, ...`; `p` itself should not be marked).

4. Find the smallest number in the list greater than `p` that is not marked. If there is no such number, stop. Otherwise let `p` now equal this new number (which is the next prime), and repeat from step 3.

5. When the algorithm terminates, the numbers remaining unmarked in the list are all the primes below or equal to `n`.

Implement the algorithm *exactly* as written above. The lab session is where we discuss possible variations and optimisations.

Naming. In the following, we call the marker array (the list in which multiples are marked) `arr`.

In task 1 you must allocate `arr` on the stack, as a local array declaration inside `print_sieves`. The total memory must not exceed `n+8` bytes; this forces you to think about the element type (the smallest type that can hold the flag is `char`, not `int`).

Task 2: Copy the content of `sieves.c` to a new file `sieves-heap.c`. Now, allocate `arr` on the heap with `malloc` instead of declaring it as a local array. Do not forget to call `free` to release the memory before your function returns.

Questions for Assignment 3 Before the oral exam, you should prepare the answers to the following questions. You will need to be able to answer these questions to pass the assignment.

- What are the main steps in the algorithm? How have you implemented these steps?
- What is the largest prime number that you can print within 2 seconds of computation? What is the largest number you can print within 10 seconds? Is it the same for *print_prime.c*, *sieves.c*, and *sieves-heap.c*? Why or why not?

3.4 Assignment 4: Pointers

The purpose of this exercise is to get a detailed understanding of pointers and pointer concepts such as dereferencing and pointer arithmetic.

Open `pointers.c` and `pointers.S` from the `lab2-files.zip` you already downloaded for Assignment 1. `pointers.c` is just a template and cannot be compiled without adding additional code; `pointers.S` is complete and may be executed in the RARS simulator. *Exception:* when assembling `pointers.S`, RARS prints the warning "`RARS does not recognize the .type directive. Ignored.`" – this is expected and harmless; you do not need to (and cannot) fix it.

Task: The file `pointers.S` contains an assembler program with two functions: `work` and `copycodes`. Your task is to translate these two assembly functions into C functions with matching signatures. The function names are fixed (so the existing code that calls them keeps working); for each function you must determine, by reading the assembly: its return type, the number of parameters it takes, the C type of each parameter, and a name for each parameter.

Hint. For the return type, look at what (if anything) is in `a0` just before the `jr ra` that ends the function. For the parameters, look at what each of `a0`, `a1`, `a2` is used for inside the function (the load/store widths tell you whether you are looking at a byte pointer or a word pointer).

The data declarations for `text1` and `text2` have already been provided in `pointers.c`. You must also declare `list1`, `list2`, and `counter` at file scope so that they correspond to the asm labels of the same name.

No array indexing. You may not use the array-subscript form `a[i]` anywhere in your C code for this assignment. The point of the exercise is to translate the explicit address arithmetic

of the assembly version (loading the base address of a list into a register and adding offsets) into the equivalent C: store a pointer with `char *p = list1;`, then *dereference* it with `*p`, and *advance* it with `p++` (or `p += 1;`). Reading and writing through that pointer should look like `*p = '!';` or `c = *p;`, not `list1[i] = '!';`. Note that the compiler treats `a[i]` and `*(a + i)` as identical, so this rule is about your *source* code, not what the compiler does internally.

Note: Your two C functions (*work* and *copycodes*) must be interchangeable with the assembly language versions. Parameters and results must match, so that the code to call your function is the same as the code to call the corresponding assembly-language function.

If your program is implemented correctly, the following should be printed when executing the program *pointers.c*:

```
list1: ASCII codes and corresponding characters.

0x054 'T' 0x068 'h' 0x069 'i' 0x073 's' 0x020 ' ' 0x069 'i' 0x073 's'
0x020 ' ' 0x061 'a' 0x020 ' ' 0x073 's' 0x074 't' 0x072 'r' 0x069 'i'
0x06E 'n' 0x067 'g' 0x02E '.'

list2: ASCII codes and corresponding characters.

0x059 'Y' 0x065 'e' 0x074 't' 0x020 ' ' 0x061 'a' 0x06E 'n' 0x06F 'o'
0x074 't' 0x068 'h' 0x065 'e' 0x072 'r' 0x020 ' ' 0x074 't' 0x068 'h'
0x069 'i' 0x06E 'n' 0x067 'g' 0x02E '.'

Count = 35

Endian experiment: 0x23,0x00,0x00,0x00
```

Questions for Assignment 4 Before the oral exam, you should prepare the answers to the following questions. You will need to be able to answer these questions to pass the assignment.

- Explain how you get the pointer addresses to the two char arrays (*text1* and *text2*) and the counter variable (*count*) in function *work()*.
- What does it mean to increment a pointer? What is the difference between incrementing the pointer that points to the ASCII text string, and incrementing the pointer that points to the integer array? In what way is the assembler code and the C code different?
- What is the difference between incrementing a pointer and incrementing a variable that a pointer points to? Explain how your code is incrementing the *count* variable.
- Explain a statement in your code where you are dereferencing a pointer. What does this mean? Explain by comparing it with the corresponding assembler code.
- Is your computer using big-endian or little-endian? How did you come to your conclusion? Is there any benefit of using either of the two alternatives?

3.5 Assignment 5: Memory Layout

The purpose of this task is to get a good understanding of how data and code are allocated in the memory. This includes local variables, global variables, pointers, and arrays.

Download and unzip the file *riscv32tests.zip* from the Canvas assignment **Lab 2 Files + Upload code**. This assignment uses the DTEK-V toolchain and the DTEK-V board. Make sure to also download *time4riscv.zip* from the Canvas assignment **Lab 1 Files + Upload code**. Unzip all the content into a single directory and run *make* and *dtekv-run main.bin* to upload

the program to the board. Consult the lecture slides (or Canvas page) to see the memory layout of the binary.

Task: Your task is to inspect the file *main.c* and to browse through the information that is shown on your host monitor. In the *main* function, a function *print_byte/word(string, addr)* is called 20 times. This help function saves a string name *str* and displays this on your local monitor. Together with the text, the address *addr* (parameter two of *print_byte/word*) is also displayed, together with the data word (32 bits) that the address *addr* points to. You should now go through all the 20 different items that are displayed, inspect the code, and be ready to answer the following questions.

Write down your answers to the questions for this assignment; the lab assistant discusses them with you during the examination. You may optionally upload your notes (e.g. a `.txt`, `.md` or `.pdf`) to the Canvas assignment **Lab 2 Files + Upload code** so you have them on the lab computer. Notes you instead bring to the session must be **handwritten on paper** – tablets and phones are not allowed during the examination.

Questions for Assignment 5 Before the oral exam, you should prepare the answers to the following questions. You will need to be able to answer these questions to pass the assignment.

- Consider *AM18*, *AM19*, and *AF1*. Explain why *gv* ends up with the incremented value, but *m* does not.
- Pointer *cp* is a character pointer that points to a sequence of bytes. What is the size of the *cp* pointer itself?
- Explain how a C string is laid out in memory. Why does the character string that *cp* points to have to be 9 bytes?
- Which addresses have *fun* and *main*? Which sections are they located in? What kind of memory are they stored in? What is the meaning of the data that these symbols points to?

Before the examination, you should also try to answer the following. When the lab-assistant performs the examination, he/she can also clarify anything that you did not understand with the following questions:

- Which addresses are variables *in* and *gv* located at?
- Variables *p* and *m* are not global variables. Where are they allocated? Which memory section is used for these variables? Why are the address numbers for *p* and *m* much larger than for *in* and *gv*?
- At print statement *AM5*, what is the address of pointer *p*, what is the value of pointer *p*, and what value is pointer *p* pointing to?
- At print statement *AM7*, what is the address of pointer *p*, what is the value of pointer *p*, and what value is pointer *p* pointing to?
- Consider *AM14* to *AM17*. Is the RISC-V processor using big-endian or little-endian? Why?

3.6 Assignment 6: Surprise Assignment

During the lab session, each pair receives a surprise assignment on the parts of Lab 2 you have prepared. The variants are randomly assigned. You solve the assignment *together as a pair* during the session and demonstrate the working result to the teacher. Questions about it are answered *individually*.

4 Version History

1.0 – First published version. 2015-09-25.

1.1 – Added a short note that all programs need to be error and warning free, before examination. 2017-09-04.

1.2 – Ported to RISC-V/32 platform. 2024-07-27.

1.3 – 2026 revision; this variant is examined at the lab session (no auto-grader). 2026.